CONTENTS

Advanced LLM context architecture IBM Technology

RAG or Long Context: How to Get Data Into a Model

Decide between retrieval and stuffing the context window using cost, recall and data size rather than fashion.

For engineers designing the data path into an LLM application · 26 July 2026

Watch the source video | Read the condensed version | Read the highlights

BEFORE YOU START

- What a context window is and that it is finite
- A basic idea of embeddings and vector search
- Comfort reading Python

Overview

Language models are frozen in time. They know the world up to their training cutoff and nothing about what happened five minutes ago, nor anything about your internal wikis or proprietary codebase. Every application that needs current or private data therefore has to solve the same problem: get the right data into the model at the right time. This is context injection.

There are two answers. RAG is the engineering answer — chunk your documents, embed them, store the vectors, search at query time, inject the best matches. Long context is the brute-force answer — put the documents into the prompt and let the model's attention mechanism find what matters. For years the second was impossible because context windows were tiny. They are not tiny any more, which is why the question is live.

The honest answer is that the two fail in different places, and which failure you can afford decides the architecture. RAG can silently fail to retrieve the answer that was sitting right there. Long context makes you pay to re-read everything on every query, dilutes attention as it grows, and hits a hard ceiling long before enterprise data does. Neither is dead; they cover different regimes.

KEY TAKEAWAYS

- ✓ Context injection is the shared problem: a frozen model needs current or private data supplied at query time.
- ✓ RAG chunks and embeds documents ahead of time, then retrieves the closest matches per query and injects them.
- ✓ Long context skips the retrieval stack entirely and lets the attention mechanism do the finding.
- ✓ Long context collapses the infrastructure — no chunking strategy, no embedding model, no vector store, no reranker, no sync.
- ✓ RAG's characteristic failure is silent: the answer existed in the data but retrieval never surfaced it, and nothing reports an error.
- ✓ RAG cannot answer questions about what is absent, because it retrieves what exists and never sees the gap between two documents.
- ✓ Long context re-pays the processing cost on every query; RAG pays it once at index time, and prompt caching only helps for static prefixes.
- ✓ Attention dilutes as context grows, so more context is not monotonically better — and enterprise data will never fit regardless.

01 The problem both approaches solve

0:00

> Models are frozen at their training cutoff and have never seen your private data. Everything else follows from that.

There is a fundamental truth about large language models: they are frozen in time. They know a great deal about the world up until their training cutoff and absolutely nothing about what happened five minutes ago. They also know nothing about your private data — your internal wikis, your proprietary codebase, your ticket history.

So any application that needs current or private information has to solve the same problem, which is worth naming precisely: context injection. How do you get the right data into the model at the right time?

Both candidate answers put the data in the same place. It ends up in the context window, as text, alongside the user's question. The difference is entirely in how it gets selected. RAG selects a small relevant subset before the call. Long context declines to select and sends everything. That is the whole disagreement, and framing it this way makes the trade-off visible: you are choosing between the risk of selecting wrongly and the cost of not selecting at all.

KEY POINTS

- → Models know nothing past their training cutoff and nothing private to you.
- → Context injection is the shared problem: right data, right time, into the prompt.
- → Both approaches deliver the data as text in the context window.
- → They differ only in whether a selection step happens before the call.
- → The trade is the risk of selecting wrongly against the cost of sending everything.

The same call, two ways of filling it

The model sees a prompt either way. The question is what went into the placeholder — four retrieved chunks, or the entire corpus.

```
# RAG: a selection step runs first
context = retrieve(question, k=5)          # ~2,000 tokens

# long context: no selection
context = open("manual.txt").read()        # ~250,000 tokens

# identical from here on
model.generate(f"Context:\n{context}\n\nQuestion: {question}")
```

02 RAG: the engineering approach

0:43

> Chunk the documents, embed them, store the vectors, search semantically at query time, inject the winners.

Ahead of time, you take the documents you want the model to have — PDFs, code files, entire books — and chunk them, breaking them into smaller pieces. Those chunks pass through an embedding model, which turns each one into a vector. The vectors go into a dedicated vector database.

Then, when a user asks a question, the system performs a semantic search to find the most relevant chunks and injects them into the context window. The window now holds the user's prompt plus the retrieved chunks, and together they form the input the model actually receives.

This works. But notice what it rests on: the hope that your retrieval logic actually found the right information. Everything downstream is conditioned on that step. The model cannot answer from a chunk it was never given, and it has no way to know that a better chunk existed. Retrieval quality is not one component's quality among several — it is a ceiling on the whole system.

KEY POINTS

- → Indexing happens ahead of time: chunk, embed, store.
- → Query time is: embed the question, search, inject the top matches.
- → The context window ends up holding the prompt plus the retrieved chunks.
- → The entire system is conditioned on retrieval having found the right chunks.
- → The model cannot know that a better chunk existed and was not retrieved.

The indexing pipeline, once per document

Four stages, all offline. The chunking strategy is a real decision — fixed size, sliding window, or recursive splitting on structure — and it is made before you know what anyone will ask.

```
def index(document, store):
    chunks = chunk(document.text, size=800, overlap=100)
    vectors = embed_batch(chunks)                  # one forward pass per chunk

    store.upsert([
        {"doc_id": document.id, "text": c, "vector": v}
        for c, v in zip(chunks, vectors)
    ])

# runs once per document, not per query — this is the whole point
```

The query path

One embedding call and one search, then an ordinary generation. The `k` is doing a lot of work: it is simultaneously your cost control and your recall ceiling.

```
def answer(question, store, k=5):
    hits = store.search(embed(question), k=k)

    context = "\n\n".join(f"[{i+1}] {h.text}" for i, h in enumerate(hits))
    return model.generate(
        f"Answer from the context only.\n\n"
        f"<context>\n{context}\n</context>\n\n{question}"
    )
```

03 Long context: the model-native approach

2:10

> Skip the database and the embedding model. Put the documents in the prompt and let attention do the heavy lifting.

The second approach is more brute force. You skip the vector database. You skip the embedding model. You take your documents and put them straight into the context window, then let the model's attention mechanism do the work of finding the answer.

This is the model-native solution in a real sense: attention is already a relevance-weighting mechanism. Every layer computes how much each part of the input should influence every other part. Retrieval is doing an approximate, external, one-shot version of something the model does internally, exactly, at every layer.

For a long time this was not an option, because context windows were too small to hold anything useful. That constraint is what created the RAG architecture in the first place — it was never chosen on its merits over long context, because long context did not exist as a choice.

KEY POINTS

- → No embedding model, no vector store, no retrieval step.
- → The attention mechanism performs relevance weighting internally, at every layer.
- → Retrieval is an approximate external version of what attention already does.
- → RAG originated as a workaround for small context windows, not as the better design.
- → The model sees everything, so there is nothing for a selection step to miss.

The no-stack stack

The same capability, minus five components. Every box removed is a box that cannot break, drift out of sync, or need capacity planning.

RAG	long context
chunking strategy	–
embedding model	–
vector database	–
reranker	–
index sync pipeline	–
retrieval logic	–
	read the files
model call	model call

The whole implementation

Not a simplification for teaching — this really is the entire data path. That is the argument in its strongest form.

```
docs = "\n\n".join(p.read_text() for p in Path("corpus").glob("*.md"))

answer = model.generate(f"{docs}\n\nQuestion: {question}")
```

04 What changed: three orders of magnitude

2:50

> Early context windows held about 4K tokens. Today's frontier models handle a million and more, which is what makes the question worth asking.

Initially context windows were tiny. Early models could hold somewhere around 4K tokens — you could not fit a novel in there, let alone a corporate knowledge base. You basically had to use RAG. There was no debate to have.

Today's models routinely handle far more, with some at a million tokens and beyond. A million tokens is roughly 700,000 words: you could fit the entire Lord of the Rings series into the prompt and still have room for The Hobbit.

That jump forces a genuine architectural question. If you can select all your documentation and paste it into the context window, do you really need the overhead of embedding models and vector stores? Is RAG an unnecessary complexity layer that outlived the constraint that produced it?

Be careful with the units when reasoning about this. Tokens are not words and not characters — roughly three-quarters of a word for English prose, and considerably worse for code, JSON, tables and non-Latin scripts, where a single word can cost several tokens. Estimating your corpus in words will understate it, sometimes by a lot. Count tokens with the actual tokenizer before concluding that everything fits.

KEY POINTS

- → Early windows around 4K tokens made RAG mandatory rather than chosen.
- → Frontier models now reach a million tokens and beyond.
- → A million tokens is roughly 700,000 words of English prose.
- → Tokens are not words: code, JSON and non-Latin text are far denser.
- → Measure your corpus with a real tokenizer before assuming it fits.

Measure, do not estimate

The rule of thumb is fine for prose and misleading for everything else. Run this over your actual corpus — teams are routinely surprised by a factor of two on codebases and structured data.

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")

total = 0
for path in Path("corpus").rglob("*"):
    if path.is_file():
        n = len(enc.encode(path.read_text(errors="ignore")))
        total += n
        print(f"{n:>9,}  {path}")

print(f"{total:>9,}  TOTAL")
print(f"fits in a 1M window: {total < 1_000_000}")
```

Density varies more than people expect

Approximate ratios, not measurements — the point is the spread. A corpus that is 'only 200,000 words' can be well past a million tokens if it is mostly JSON.

```
English prose      ~0.75 words per token   → 1M tokens ≈ 700k words
source code        denser: punctuation and identifiers split aggressively
JSON / tables      denser still: braces, quotes and keys repeat constantly
non-Latin scripts  often several tokens per character
```

05 The case for long context: no stack, and no retrieval lottery

4:20

> Two arguments. You delete most of the architecture, and you delete the failure mode that never announces itself.

The first argument is infrastructure collapse. A production RAG system is heavy. You need a chunking strategy — fixed size, sliding window, recursive, you decide, and the decision matters. You need an embedding model to encode the data. You need a vector database to store it. You need a reranker to sort the results. You need to keep all the vectors in sync with the source data. That is a lot of moving parts and a lot of places for things to break. Long context removes the database, the embeddings and the retrieval logic; the architecture simplifies down to fetching the data and sending it.

The second argument is sharper. RAG introduces a critical point of failure at the retrieval step itself. Semantic search compares vectors and returns the closest matches, and it is probabilistic. For all sorts of reasons it can fail to surface the relevant document. This has a name: silent failure. The answer existed in the data, but the model never saw it, because retrieval did not return it.

Silent is the operative word. Nothing raises an exception. No error is logged. The model receives four plausible-looking chunks, none of which contain the answer, and produces a confident response built from them. From the outside it looks exactly like a model that does not know something. With long context there is no retrieval step, so this failure mode simply does not exist — the model gets to see everything.

KEY POINTS

- → A production RAG system is chunking, embedding, storage, reranking and sync — five things that can break.
- → Long context reduces the data path to reading files and calling the model.
- → Semantic search is probabilistic and can miss documents that are present.
- → Silent failure: the answer existed, retrieval missed it, nothing reported an error.
- → With no retrieval step, long context cannot fail this way at all.

What a silent failure looks like from the outside

This is the entire problem. Both paths return a fluent answer and a 200 status. The only way to tell them apart is to already know the answer — which is why silent failure needs a deliberate measurement rather than monitoring.

```
question: "What is the escalation path for a P1 outage after midnight?"

retrieved:
  [1] "Escalation policy overview ..."        ← topical, wrong section
  [2] "P2 and P3 handling during business..." ← topical, wrong severity
  [3] "On-call rotation calendar ..."         ← topical, no procedure

answer: fluent, confident, wrong
logs:   no errors
status: 200 OK

# the correct paragraph was chunk #4,912 and scored 6th
```

Measuring recall so failure stops being silent

Build a set of questions whose answers you can point at in the corpus, then check whether retrieval actually returns them. This converts an invisible failure into a number you can watch. Without it you have no idea what your ceiling is.

```
GOLD = [
    ("escalation path for P1 after midnight", "runbook.md#after-hours"),
    ("how do we rotate the signing key",      "security.md#key-rotation"),
]

def recall_at_k(store, k):
    hits = sum(
        any(h.source == expected for h in store.search(embed(q), k=k))
        for q, expected in GOLD
    )
    return hits / len(GOLD)

for k in (1, 3, 5, 10, 25):
    print(f"recall@{k:<3} {recall_at_k(store, k):.0%}")

# whatever this tops out at is the ceiling on your whole system
```

06 The whole-book problem: reasoning about what is absent

5:43

> RAG retrieves what exists. Some questions are about what is missing, and no snippet contains a gap.

This is the most interesting argument against retrieval, because it is structural rather than a matter of tuning. RAG is fundamentally designed to retrieve what exists. It finds a semantic match between your query and a specific snippet of text in the database.

So consider a corpus with a product requirements document and a set of release notes, and the question: which security requirements were omitted from the final release? Query for omitted security requirements and the vector search looks for chunks discussing security and requirements. It retrieves snippets from the requirements doc. It retrieves snippets from the release notes. But it cannot retrieve the gap between them, because the gap is not text and has no embedding.

Because RAG shows the model only a few isolated snapshots, the model never sees the full picture needed to spot what is missing. Answering the question requires holding both documents in full and comparing them — which is exactly what long context does by putting the whole requirements doc and the whole release notes into the window.

The generalisation is worth carrying around: retrieval is a poor fit whenever the answer is a property of the corpus as a whole rather than a passage within it. Absences, counts, exhaustive comparisons, 'how many', 'which ones never', 'what is inconsistent between'. If no single passage could contain the answer, top-k retrieval cannot find it, no matter how good the embeddings are.

KEY POINTS

- → RAG matches queries to passages, and a gap is not a passage.
- → Questions about omissions require seeing both documents in full.
- → Isolated snippets cannot support a global comparison.
- → The same limit applies to counting, exhaustiveness and cross-document consistency.
- → If no single passage could contain the answer, top-k retrieval structurally cannot find it.

Why the retrieval succeeds and the answer still fails

Retrieval did its job here — the chunks are genuinely relevant. That is what makes this failure different from silent failure and impossible to fix by improving the embeddings.

```
question: "which security requirements were omitted from the final release?"

retrieved:
  [1] requirements.md  "3.4 All API endpoints shall enforce rate limiting..."
  [2] requirements.md  "3.7 Session tokens shall expire after 30 minutes..."
  [3] release-notes.md "Added rate limiting to public endpoints."
  [4] release-notes.md "Improved session handling performance."

# every chunk is relevant. the answer — that 3.7 was never implemented —
# is in the *absence* of a matching release note, which no chunk contains.
```

The question shape that decides the architecture

A quick test before choosing. If you can imagine highlighting a passage that answers the question, retrieval can work. If the answer only emerges from surveying everything, it cannot.

```
answerable from one passage        → retrieval works
  "what is the session timeout?"
  "how do I rotate the signing key?"

answerable only from the whole     → needs the full documents
  "which requirements were dropped?"
  "how many endpoints lack rate limiting?"
  "where do these two specs contradict each other?"
  "summarise the argument of this book"
```

07 The case for RAG: you pay to re-read, every single time

7:53

> Long context creates a compute inefficiency that scales with traffic. RAG pays the processing cost once, at indexing time.

Take a 500-page manual. As tokens that is somewhere around 250,000. With long context you put it in the prompt, which means the model processes that entire manual on every single user query. Ten thousand queries, ten thousand full readings of the same unchanged document.

RAG also has to process that manual, but it pays that cost once, at indexing time. After that, each query processes a few thousand tokens of retrieved chunks. The asymmetry compounds with traffic: long context's cost is proportional to corpus size times query volume, while RAG's is corpus size plus a small amount times query volume.

Prompt caching partially offsets this. If the long prefix is identical across requests, providers can cache the processed representation and charge less for the cached portion. That genuinely helps for static data. But it depends on the prefix being byte-identical and on the cache still being warm, and it does nothing for dynamic data streams where content changes frequently — there you are stuck paying the full tax on every request. It also constrains your prompt layout: anything that varies must come after everything that does not, or the cache misses on every call.

KEY POINTS

- A 500-page manual is roughly 250,000 tokens, re-processed on every long-context query.
- RAG pays the processing cost once at index time, then a few thousand tokens per query.
- Long-context cost scales as corpus size × query volume; RAG as corpus size + small × volume.
- Prompt caching offsets this only for stable, byte-identical prefixes that stay warm.
- Frequently changing content invalidates the cache and restores the full cost.

Where the two curves cross

Rates are placeholders — substitute your provider's current numbers, since they change. What does not change is the shape: long context is dominated by a term that multiplies corpus size by traffic, and RAG is not.

```
CORPUS_TOKENS = 250_000
RETRIEVED     = 3_000
IN_RATE       = 3.00 / 1_000_000     # per input token
EMBED_RATE    = 0.02 / 1_000_000

def long_context(queries, cache_hit_rate=0.0):
    effective = CORPUS_TOKENS * (1 - cache_hit_rate * 0.9)  # cached ≈ 10% cost
    return queries * effective * IN_RATE

def rag(queries):
    index_once = CORPUS_TOKENS * EMBED_RATE
    return index_once + queries * RETRIEVED * IN_RATE

for q in (100, 1_000, 10_000, 100_000):
    print(f"{q:>7,}  long={long_context(q):>9.2f}  "
          f"cached={long_context(q, 0.9):>8.2f}  rag={rag(q):>8.2f}")
```

Prompt caching dictates prompt layout

Caching keys on an exact prefix. Put anything variable — a timestamp, a user ID, the question — before the bulk content and every request misses. This ordering is the single most common reason teams see no benefit from caching they believe they enabled.

```
# cache misses every time: the prefix changes on every request
prompt = f"Time: {now()}\nUser: {user_id}\n\n{manual}\n\n{question}"

# cache hits: the stable bulk comes first, variables last
prompt = f"{manual}\n\nTime: {now()}\nUser: {user_id}\n\n{question}"
#          └─── cacheable prefix ──┘└──── varies per request ────┘
```

08 The case for RAG: attention dilutes as context grows

8:36

> There is an intuitive assumption that data in the window will be used. Research suggests otherwise.

The assumption is that if the data is in the context window, the model will probably use it. In practice that holds less well than it sounds. As the window grows — a hundred thousand tokens, five hundred thousand — the attention mechanism gets diluted. Ask a specific question about a single paragraph buried in the middle of a two-thousand-page document and the model often fails to surface it, or hallucinates details from the surrounding text.

RAG inverts this by giving the model less. Retrieve the top five relevant chunks and you have removed the haystack and handed over just the needles. The model is forced to focus on signal rather than noise, and precision on that specific question typically goes up rather than down.

This is why 'more context is better' is wrong as a general rule. Context is not free even when it fits and even when you are willing to pay for it: irrelevant material competes for attention with relevant material. Which produces a genuinely counter-intuitive practice — pruning context can improve accuracy, and if you are not measuring, you will not notice that your 400,000-token prompt performs worse than the 8,000-token version of the same request.

So test it rather than assuming either way. Build a needle-in-a-haystack evaluation over your own corpus and your own question types, place known facts at varying depths, and measure retrieval accuracy as a function of context length and position. The behaviour differs between models and changes between versions.

KEY POINTS

- → Attention dilutes as context length grows; presence in the window does not guarantee use.
- → Facts buried mid-document are the hardest to surface, and failures can look like hallucination.
- → Retrieving the top chunks removes the haystack and hands the model the needles.
- → More context can lower accuracy — irrelevant tokens compete with relevant ones.
- → Measure position and length effects on your own data; behaviour varies by model and version.

A needle-in-a-haystack test for your own corpus

Insert a fact the model cannot know, at a controlled depth, then ask for it. Sweep length and depth to map where your accuracy falls off — that boundary is what tells you how much context you can actually rely on.

```
NEEDLE = "The internal codename for the Q3 migration is BLUE HERON."
QUESTION = "What is the internal codename for the Q3 migration?"

def trial(filler_tokens, depth_fraction):
    filler = make_filler(filler_tokens)
    cut = int(len(filler) * depth_fraction)
    haystack = filler[:cut] + NEEDLE + filler[cut:]

    out = model.generate(f"{haystack}\n\n{QUESTION}")
    return "BLUE HERON" in out.upper()

for length in (10_000, 50_000, 200_000, 500_000):
    row = [trial(length, d) for d in (0.1, 0.3, 0.5, 0.7, 0.9)]
    print(f"{length:>7,}  " + "  ".join("ok " if r else "MISS" for r in row))
```

Less context, better answer

The same question against the same corpus, two ways. This is the result people find hardest to believe until they measure it on their own data.

```
full document in context     : 480,000 tokens
  → answer draws on a similar passage 200 pages away, states the wrong figure

top-5 retrieved chunks       :   3,200 tokens
  → answer quotes the correct paragraph

# 150x less input, and correct instead of wrong
```

09 The case for RAG: the infinite dataset

9:19

> A million-token window sounds enormous until you compare it to an enterprise data lake measured in terabytes.

A context window of millions of tokens sounds like a lot. In the scheme of enterprise data it is a drop in the bucket. An enterprise data lake is measured in terabytes, sometimes petabytes. There is no context window on any roadmap that closes that gap, and the gap is growing at least as fast as window sizes are.

So if you need a system that can answer over everything the organisation knows, you need a retrieval layer that filters the corpus down to something that fits. That is not a limitation of the current generation of models to be waited out — it is a structural relationship between how much data organisations accumulate and how much any single forward pass can process.

The useful reframing is that this argument is about the shape of your data, not its absolute size. A bounded corpus — one contract, one book, one codebase, one incident's logs — can fit, and the long-context arguments apply in full. An unbounded corpus cannot, will not, and the question of whether it fits today is the wrong question, because the answer changes with every document anyone adds.

KEY POINTS

- → Enterprise data lakes are measured in terabytes or petabytes; windows are measured in millions of tokens.
- → The gap is structural and is not closing.
- → Answering over everything requires a filtering layer that reduces the corpus to what fits.
- → The deciding property is whether the corpus is bounded, not how large it is right now.
- → A bounded corpus that fits today may not be bounded — check growth, not just current size.

Bounded or unbounded is the real question

Ask what the corpus looks like in two years rather than what it looks like now. Anything that grows with usage — tickets, logs, messages, commits — is unbounded and needs retrieval regardless of today's measurement.

bounded, fits, stays fitting	unbounded, will never fit
one legal contract	every contract the firm has
one book	the company wiki
one service's source	the monorepo
one incident's logs	all logs, all time
a product spec + release notes	the ticket system

The hybrid most production systems converge on

Retrieval as a coarse filter rather than a precise selector. Retrieve generously to make silent failure unlikely, then let attention do the fine-grained work on a corpus small enough that dilution has not set in. This captures most of both arguments.

```
def answer(question, store):
    # coarse: cut terabytes down to something that fits, with slack
    candidates = store.search(embed(question), k=60)

    # fine: let attention pick within a manageable window
    context = "\n\n".join(c.text for c in candidates)   # ~40k tokens

    return model.generate(
        f"<context>\n{context}\n</context>\n\nQuestion: {question}"
    )

# high k → silent failure becomes unlikely
# bounded context → attention stays sharp
```

10 Choosing

10:05

> Bounded data plus global reasoning favours long context. Unbounded enterprise knowledge still requires a vector database.

If your problem involves a bounded dataset and requires complex global reasoning — analysing a specific legal contract, summarising a book, comparing two specifications in full — long context is the way to go. It simplifies the stack and it improves the reasoning, because the model sees the whole object rather than fragments of it.

If you are navigating the effectively infinite dataset of enterprise knowledge, the vector database remains the only viable warehouse. There is nothing to debate; the data does not fit.

Most real systems land between these, which is why the hybrid pattern has become standard: retrieve broadly to reduce an unbounded corpus to a bounded one, then hand the model enough context to reason globally over what survived. You get retrieval's scalability with much of long context's robustness, and you tune one dial — how much you retrieve — against measured recall and measured dilution.

Whichever you choose, the decision is only defensible if you measured. Recall on a gold set tells you what retrieval is missing. A needle test tells you where attention stops being reliable. Cost per query at your actual traffic tells you what the architecture bills. Without those three numbers you are choosing by fashion, and both fashions have been wrong within the last two years.

KEY POINTS

- → Bounded corpus plus global reasoning → long context.
- → Unbounded enterprise knowledge → retrieval, non-negotiably.
- → The common middle ground is retrieve-broadly-then-reason, tuned by k.
- → Three measurements decide it: recall on a gold set, needle accuracy by length, cost at real traffic.
- → Both defaults have been wrong recently; measure rather than following the trend.

The decision, on one screen

Work down the column that matches your corpus. Most of the disagreement about RAG comes from people arguing from different rows without saying which.

	long context	RAG
corpus	bounded	unbounded
question type	global / comparative	passage-level
cost driver	tokens x every query	one-off indexing
characteristic failure	attention dilution	silent retrieval miss
infrastructure	none	embed + store + sync
freshness	re-read the file	re-index the document
answers "what is missing?"	yes	no
scales to a data lake	no	yes

The three numbers to collect before deciding

Cheap to produce and they end the argument. Teams that skip them tend to re-litigate the architecture every six months as the fashion swings.

```
1. recall@k on 30–50 gold questions with known source passages
     → the ceiling on any RAG design you build

2. needle accuracy at your realistic context lengths and depths
     → where long context stops being trustworthy for your data

3. cost and p95 latency per query at projected traffic, both ways
     → which one you can actually afford to run
```

Glossary

Context injection

Getting the right external data into a model's prompt at query time, so it can answer about things outside its training data.

Context window

The maximum number of tokens a model can process in one call, covering the prompt and the generated output together.

Chunking

Splitting documents into smaller passages before embedding. Strategies include fixed size, sliding window and recursive splitting on structure.

Embedding model

A model that converts text into a vector whose position encodes meaning, so similar passages sit near each other.

Vector database

Storage built for fast nearest-neighbour search over embeddings, plus the indexing and sync machinery around it.

Semantic search

Ranking documents by vector similarity to the query rather than by shared keywords, so meaning matches even when wording differs.

Silent failure

When retrieval misses a document that contained the answer. No error is raised — the model simply answers from the wrong material, confidently.

Reranker

A second-stage model that re-scores retrieved candidates more accurately than vector similarity, usually over a shortlist.

Long context

Placing whole documents directly in the prompt and letting the attention mechanism locate what matters, with no retrieval step.

Prompt caching

Reusing a provider's processed representation of an identical prompt prefix across requests, cutting the cost of the repeated portion.

Needle in a haystack

A test that plants a known fact at a controlled depth in a long context and checks whether the model can retrieve it.

Attention dilution

The degradation in a model's ability to focus on a specific passage as the surrounding context grows.

recall@k

The fraction of questions for which the correct source passage appears in the top k retrieved results. The ceiling on a RAG system's accuracy.

Token

The unit a model actually processes — a word fragment. Roughly three-quarters of a word for English prose, and much denser for code or JSON.

Check yourself

Answer first, then open each one.

Why is a retrieval miss described as a silent failure, and what does that imply about monitoring?

Because nothing errors: retrieval returns plausible chunks, the model answers fluently from them, and the response is a 200 with no warning. Standard monitoring cannot see it, since there is no failure signal to alert on. The only way to detect it is a gold set of questions with known source passages and a recall@k measurement run deliberately.

Retrieval returns four chunks that are all genuinely relevant, yet the system still cannot answer 'which requirements were dropped from the release?'. Why is better embedding not the fix?

The answer is an absence, and an absence is not a passage — there is no text anywhere in the corpus that says a requirement was omitted, so no embedding can point at it. The question requires holding both documents in full and comparing them. This is a structural limit of passage retrieval, not a quality problem.

A team enables prompt caching for a long-context application and sees no cost reduction. What is the most likely cause?

Something variable sits at the front of the prompt — a timestamp, a session ID, the question itself — so the cacheable prefix differs on every request and the cache misses every time. Caching keys on an exact prefix, so all stable bulk content must come first and everything variable must come last.

Why can adding more relevant-looking context to a prompt make answers worse rather than better?

Attention dilutes as context grows: irrelevant material competes with relevant material for the model's focus, and a fact buried mid-document can be missed or blended with surrounding text. This is why retrieving the top few chunks often beats supplying everything, even when everything fits and you are willing to pay for it.

Your corpus is 400,000 tokens today and fits comfortably in the window. Why might retrieval still be the right architecture?

Because the deciding property is whether the corpus is bounded, not its size right now. If it grows with usage — tickets, logs, commits, messages — today's fit is temporary, and rebuilding the data path later is far more expensive than building retrieval now. Ask what it looks like in two years.

Why does the hybrid pattern — retrieve fifty chunks, then reason over all of them — capture most of both arguments?

A high k makes silent failure unlikely, because the correct passage almost certainly appears somewhere in a wide net. Bounding the result to a moderate context keeps attention sharp and cost predictable, while letting the model do global reasoning over what survived. Retrieval acts as a coarse filter rather than a precise selector, which is the job it is actually good at.

Where to go next

- → Build a gold set of 30 questions with known source passages for your own corpus and plot recall@k for k in 1, 3, 5, 10, 25, 50 — the plateau is the ceiling on any RAG system you build on it.
- → Run the needle-in-a-haystack script against the model you actually use, sweeping length and depth, and find where accuracy falls off for your content type rather than for generic prose.
- → Compute the crossover point where long-context cost exceeds RAG cost at your real query volume, then recompute it with a realistic prompt-cache hit rate.
- → Take a question your RAG system answers badly and re-ask it with the full documents in context; if it becomes correct, you have found a whole-book problem rather than a retrieval-quality problem.
- → Implement the retrieve-60-then-reason hybrid and sweep k against both recall and answer accuracy — the two curves usually peak at different values, and the gap is informative.
- → Instrument your RAG pipeline to log the retrieved chunk IDs alongside every answer, so that when a wrong answer is reported you can tell immediately whether retrieval or generation was at fault.
